

LAPORAN PRAKTIKUM
STRUKTUR DATA
SINGLE LINKED LIST



Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang Single Linked List menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa lapporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Single Linked List.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.1.1 Konsep Dasar	2
2.1.2 Operasi Utama.....	2
2.2 Kode Program	2
2.2.1 Class NodeSLL_2511533008	2
2.2.2 Class TambahSLL_2511533008	3
2.2.3 Class PencarianSLL_2511533008	7
2.2.4 Class HapusSLL_2511533008.....	9
2.3 Output Program.....	13
2.3.1 Output TambahSLL_2511533008.....	13
2.3.2 Output PencarianSLL_2511533008.....	14
2.3.3 Output HapusSLL_2511533008	14
BAB III PENUTUP	15
3.1 Kesimpulan	15
DAFTAR PUSTAKA	16

TUGAS	17
1. Class Pasien_2511533008.....	17
2. Class RumahSakit_2511533008	17
3. Output Tugas	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Salah satu bentuk struktur data yang banyak digunakan adalah Single Linked List (SLL), yaitu kumpulan simpul (node) yang saling terhubung secara sekuensial melalui pointer. Berbeda dengan array yang memiliki ukuran tetap, Single Linked List bersifat dinamis sehingga memungkinkan penambahan dan penghapusan data secara lebih fleksibel. Melalui praktikum ini, kita diharapkan memahami konsep dasar node, head, dan pointer, serta mampu mengimplementasikan operasi dasar seperti traversal, insertion, deletion, dan searching. Pemahaman terhadap Single Linked List tidak hanya penting untuk memperkuat logika pemrograman, tetapi juga menjadi landasan dalam mempelajari struktur data yang lebih kompleks seperti double linked list, stack, queue, maupun tree.

1.2 Tujuan

1. Memahami konsep dasar Single Linked List (SLL) sebagai salah satu struktur data dinamis.
2. Mengenali komponen utama linked list: node, head, dan pointer.
3. Mempelajari dan mengimplementasikan operasi dasar: traversal, insertion, deletion, dan searching.

1.3 Manfaat

1. Memahami konsep dasar Single Linked List (SLL) sebagai salah satu struktur data dinamis.
2. Dapat mengenali komponen utama linked list: node, head, dan pointer.
3. Bisa mengimplementasikan operasi dasar: traversal, insertion, deletion, dan searching

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Konsep Dasar

Single Linked List merupakan struktur data yang terdiri dari simpul (node) yang saling terhubung satu arah.

Node : setiap simpul memiliki dua bagian:

1. Data → menyimpan nilai/informasi
2. Pointer (next) → menunjuk ke simpul berikutnya

Head : simpul pertama dalam list.

Tail : simpul terakhir, pointer next-nya bernilai null.

2.1.2 Operasi Utama

Traversal (Penelusuran) :

- Menelusuri node dari head hingga null.
- Digunakan untuk menampilkan isi list.

Insertion (Penyisipan) :

- Di depan (insert at front) : node baru menjadi head.
- Di belakang (insert at end) : node baru ditambahkan setelah tail.
- Di posisi tertentu : node baru disisipkan di antara node lain.

Deletion (Penghapusan):

- Head : menghapus simpul pertama.
- Tail : menghapus simpul terakhir.
- Posisi tertentu : menghapus simpul di Lokasi tertentu.

Searching (Pencarian) : menelusuri list untuk menemukan apakah suatu nilai ada.

2.2 Kode Program

2.2.1 Class NodeSLL_2511533008

Kode Program:

```

package pekan5_2511533008;

public class NodeSLL_2511533008 {
    // node bagian data
    int data_3008;
    // pointer ke node berikutnya
    NodeSLL_2511533008 next_3008;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533008(int data_3008) {
        this.data_3008 = data_3008;
        this.next_3008 = null;
    }
}

```

Penjelasan :

1. int data_3008; menyimpan data/nilai dari node.
2. NodeSLL_2511533008 next_3008; pointer ke node selanjutnya.
3. Konstruktor

```

public NodeSLL_2511533008(int data_3008) {
    this.data_3008 = data_3008;
    this.next_3008 = null;
}

```

Gambar 2.2.1.1 konstruktor

2.2.2 Class TambahSLL_2511533008

Kode Program:

```

package pekan5_2511533008;

public class TambahSLL_2511533008 {
    public static NodeSLL_2511533008
insertAtFront_3008(NodeSLL_2511533008 head_3008, int value_3008) {
    NodeSLL_2511533008 new_node_3008 = new
NodeSLL_2511533008(value_3008);
    new_node_3008.next_3008 = head_3008;
    return new_node_3008;
}
    // fungsi menambahkan node di akhir sll
    public static NodeSLL_2511533008
insertAtEnd_3008(NodeSLL_2511533008 head_3008, int value_3008) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511533008 newNode_3008 = new
NodeSLL_2511533008(value_3008);
    //jika list kosong maka node jadi head
    if (head_3008 == null) {
        return newNode_3008;
    }
}
}

```

```

    }
    // simpan head ke variabel sementara
    NodeSLL_2511533008 last_3008 = head_3008;
    // telusuri ke node akhir
    while (last_3008.next_3008 != null) {
        last_3008 = last_3008.next_3008;
    }
    // ubah pointer
    last_3008.next_3008 = newNode_3008;
    return head_3008;
}

static NodeSLL_2511533008 GetNode_3008(int data_3008) {
    return new NodeSLL_2511533008(data_3008);
}

static NodeSLL_2511533008 insertPos_3008(NodeSLL_2511533008
headNode_3008, int position_3008, int value_3008) {
    NodeSLL_2511533008 head_3008 = headNode_3008;
    if (position_3008 < 1)
        System.out.print("Invalid position");
    if (position_3008 == 1) {
        NodeSLL_2511533008 new_node_3008 = new
NodeSLL_2511533008(value_3008);
        new_node_3008.next_3008 = head_3008;
        return new_node_3008;
    } else {
        while (position_3008-- != 0) {
            if (position_3008 == 1) {
                NodeSLL_2511533008 newNode_3008 =
GetNode_3008(value_3008);
                newNode_3008.next_3008 =
headNode_3008.next_3008;
                headNode_3008.next_3008 =
newNode_3008;
                break;
            }
            headNode_3008 = headNode_3008.next_3008;
        }
        if (position_3008 != 1)
            System.out.print("Posisi di luar
jangkauan");
        return head_3008;
    }
}

public static void printList_3008(NodeSLL_2511533008
head_3008) {
    NodeSLL_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        System.out.print(curr_3008.data_3008 + "--
>");
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008.next_3008 == null) {
        System.out.print(curr_3008.data_3008);
    }
}

```



```

        System.out.println();
    }
}

public static void main(String[] args) {
    // buat linked list 2->3->5->6
    NodeSLL_2511533008 head_3008 = new
NodeSLL_2511533008(2);
    head_3008.next_3008 = new NodeSLL_2511533008(3);
    head_3008.next_3008.next_3008 = new
NodeSLL_2511533008(5);
    head_3008.next_3008.next_3008.next_3008 = new
NodeSLL_2511533008(6);
    //cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList_3008(head_3008);
    //tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data_3008 = 1;
    head_3008 = insertAtFront_3008(head_3008, data_3008);
    //cetak update list
    printList_3008(head_3008);
    //tambahkan node baru dibelakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_3008 = 7;
    head_3008 = insertAtEnd_3008(head_3008, data2_3008);
    //cetak update list
    printList_3008(head_3008);
    System.out.print("tambah 1 simpul ke data 4: ");
    int data3_3008 = 4;
    int pos_3008 = 4;
    head_3008 =
insertPos_3008(head_3008,pos_3008,data3_3008);
    //cetak update list
    printList_3008(head_3008);
}
}

```

Penjelasan:

1. Fungsi insertAtFront_3008

```

public static NodeSLL_2511533008 insertAtFront_3008(NodeSLL_2511533008 head_3008, int value_3008) {
    NodeSLL_2511533008 new_node_3008 = new NodeSLL_2511533008(value_3008);
    new_node_3008.next_3008 = head_3008;
    return new_node_3008;
}

```

Gambar 2.2.2.1 fungsi 1

Fungsi ini digunakan untuk menambahkan node baru di bagian depan list.

2. Fungsi insertAtEnd_3008

```
// fungsi menambahkan node di akhir sll
public static NodeSLL_2511533008 insertAtEnd_3008(NodeSLL_2511533008 head_3008, int value_3008) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511533008 newNode_3008 = new NodeSLL_2511533008(value_3008);
    //jika list kosong maka node jadi head
    if (head_3008 == null) {
        return newNode_3008;
    }
    // simpan head ke variabel sementara
    NodeSLL_2511533008 last_3008 = head_3008;
    // telusuri ke node akhir
    while (last_3008.next_3008 != null) {
        last_3008 = last_3008.next_3008;
    }
    // ubah pointer
    last_3008.next_3008 = newNode_3008;
    return head_3008;
}
```

Gambar 2.2.2.2 fungsi 2

Fungsi ini digunakan untuk menambahkan node baru ke bagian belakang list.

3. Fungsi insertPos_3008

```
static NodeSLL_2511533008 insertPos_3008(NodeSLL_2511533008 headNode_3008, int position_3008, int value_3008) {
    NodeSLL_2511533008 head_3008 = headNode_3008;
    if (position_3008 < 1)
        System.out.print("Invalid position");
    if (position_3008 == 1) {
        NodeSLL_2511533008 newNode_3008 = new NodeSLL_2511533008(value_3008);
        newNode_3008.next_3008 = head_3008;
        return newNode_3008;
    } else {
        while (position_3008-- != 0) {
            if (position_3008 == 1) {
                NodeSLL_2511533008 newNode_3008 = GetNode_3008(value_3008);
                newNode_3008.next_3008 = headNode_3008.next_3008;
                headNode_3008.next_3008 = newNode_3008;
                break;
            }
            headNode_3008 = headNode_3008.next_3008;
        }
        if (position_3008 != 1)
            System.out.print("Posisi di luar jangkauan");
        return head_3008;
    }
}
```

Gambar 2.2.2.3 fungsi 3

Fungsi ini digunakan untuk menambahkan node baru pada posisi tertentu.

4. Fungsi printList_3008

```
public static void printList_3008(NodeSLL_2511533008 head_3008) {
    NodeSLL_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        System.out.print(curr_3008.data_3008 + "-->");
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008.next_3008 == null) {
        System.out.print(curr_3008.data_3008);
        System.out.println();
    }
}
```

Gambar 2.2.2.4 fungsi 4

Fungsi ini digunakan untuk mencetak nilai/data yang ada dalam list.

5. Algoritma utama

```

public static void main(String[] args) {
    // buat linked list 2->3->5->6
    NodeSLL_2511533008 head_3008 = new NodeSLL_2511533008(2);
    head_3008.next_3008 = new NodeSLL_2511533008(3);
    head_3008.next_3008.next_3008 = new NodeSLL_2511533008(5);
    head_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(6);
    //cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList_3008(head_3008);
    //tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data_3008 = 1;
    head_3008 = insertAtFront_3008(head_3008, data_3008);
    //cetak update list
    printList_3008(head_3008);
    //tambahkan node baru dibelakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_3008 = 7;
    head_3008 = insertAtEnd_3008(head_3008, data2_3008);
    //cetak update list
    printList_3008(head_3008);
    System.out.print("tambah 1 simpul ke data 4: ");
    int data3_3008 = 4;
    int pos_3008 = 4;
    head_3008 = insertPos_3008(head_3008, pos_3008, data3_3008);
    //cetak update list
    printList_3008(head_3008);
}
}

```

Gambar 2.2.2.5 algoritma utama

Pada algoritma utama inilah kita menggunakan fungsi fungsi yang telah kita buat tadi. Kita bisa menambahkan elemen di depan, belakang ataupun di posisi tertentu pada list, setelah itu kita bisa mencetak listnya.

2.2.3 Class PencarianSLL_2511533008

Kode Program:

```

package pekan5_2511533008;

public class PencarianSLL_2511533008 {
    static boolean searchKey_3008(NodeSLL_2511533008 head_3008,
    int key_3008) {
        NodeSLL_2511533008 curr_3008 = head_3008;
        while(curr_3008 != null) {
            if (curr_3008.data_3008 == key_3008)
                return true;
            curr_3008 = curr_3008.next_3008;
        } return false;
    }
    public static void traversal_3008(NodeSLL_2511533008
    head_3008) {
        //mulai dari head
        NodeSLL_2511533008 curr_3008 = head_3008;
    }
}

```

```

        //telusuri sampai pointer null
        while(curr_3008 != null) {
            System.out.print(" " + curr_3008.data_3008);
            curr_3008 = curr_3008.next_3008;
        } System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511533008 head_3008 = new
NodeSLL_2511533008(14);
        head_3008.next_3008 = new NodeSLL_2511533008(21);
        head_3008.next_3008.next_3008 = new
NodeSLL_2511533008(13);
        head_3008.next_3008.next_3008.next_3008 = new
NodeSLL_2511533008(30);
        head_3008.next_3008.next_3008.next_3008.next_3008 = new
NodeSLL_2511533008(10);
        System.out.print("Penelusuran SLL: ");
        traversal_3008(head_3008);
        //data yang akan dicari
        int key_3008 = 30;
        System.out.print("cari data " + key_3008 + " = ");
        if (searchKey_3008(head_3008, key_3008))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}

```

Penjelasan:

1. Fungsi searchKey_3008

```

static boolean searchKey_3008(NodeSLL_2511533008 head_3008, int key_3008) {
    NodeSLL_2511533008 curr_3008 = head_3008;
    while(curr_3008 != null) {
        if (curr_3008.data_3008 == key_3008)
            return true;
        curr_3008 = curr_3008.next_3008;
    } return false;
}

```

Gambar 2.2.3.1 fungsi 1

Fungsi ini bertujuan untuk mencari apakah sebuah nilai (key_3008) ada di dalam linked list.

2. Fungsi traversal_3008

```

public static void traversal_3008(NodeSLL_2511533008 head_3008) {
    //mulai dari head
    NodeSLL_2511533008 curr_3008 = head_3008;
    //telusuri sampai pointer null
    while(curr_3008 != null) {
        System.out.print(" " + curr_3008.data_3008);|
        curr_3008 = curr_3008.next_3008;
    } System.out.println();
}

```

Gambar 2.2.3.2 fungsi 2

Fungsi ini bertujuan untuk mencetak isi linked list dari awal sampai akhir.

3. Algoritma utama

```

public static void main(String[] args) {
    NodeSLL_2511533008 head_3008 = new NodeSLL_2511533008(14);
    head_3008.next_3008 = new NodeSLL_2511533008(21);
    head_3008.next_3008.next_3008 = new NodeSLL_2511533008(13);
    head_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(30);
    head_3008.next_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(10);
    System.out.print("Penelusuran SLL: ");
    traversal_3008(head_3008);
    //data yang akan dicari
    int key_3008 = 30;
    System.out.print("cari data " + key_3008 + " = ");
    if (searchKey_3008(head_3008, key_3008))
        System.out.println("ketemu");
    else
        System.out.println("tidak ada");
}
}

```

Gambar 2.2.3.3 algoritma utama

Pada algoritma utama pertama kita akan membuat linked list manual, lalu memanggil traversal_3008 untuk menampilkan isi list, kemudian mencari nilai 30 dengan searchKey_3008 dan jika ada maka akan tercetak “Ketemu”, jika tidak ada maka akan tercetak “tidak ada”.

2.2.4 Class HapusSLL_2511533008

Kode Program:

```

package pekan5_2511533008;

public class HapusSLL_2511533008 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511533008 deleteHead_3008(NodeSLL_2511533008 head_3008) {
        //jika SLL kosong
        if (head_3008 == null)
            return null;
        //pindahkan head ke node berikutnya
    }
}

```

```

        head_3008 = head_3008.next_3008;
        //return head baru
        return head_3008;
    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533008
removeLastNode_3008(NodeSLL_2511533008 head_3008) {
    //jika list kosong, return null
    if (head_3008 == null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_3008.next_3008 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511533008 secondLast_3008 = head_3008;
    while (secondLast_3008.next_3008.next_3008 != null) {
        secondLast_3008 = secondLast_3008.next_3008;
    }
    //hapus node terakhir
    secondLast_3008.next_3008 = null;
    return head_3008;
}
    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533008 deleteNode_3008(NodeSLL_2511533008
head_3008, int position_3008) {
        NodeSLL_2511533008 temp_3008 = head_3008;
        NodeSLL_2511533008 prev_3008 = null;
        //jika linked list null
        if (temp_3008 == null)
            return head_3008;
        //kasus 1: head dihapus
        if (position_3008 == 1) {
            head_3008 = temp_3008.next_3008;
            return head_3008;
        }
        //kasus 2: menghapus node ditengah
        //telusuri ke node yang dihapus
        for (int i_3008 = 1; temp_3008 != null && i_3008 <
position_3008; i_3008++) {
            prev_3008 = temp_3008;
            temp_3008 = temp_3008.next_3008;
        }
        //jika ditemukan, hapus node
        if (temp_3008 != null) {
            prev_3008.next_3008 = temp_3008.next_3008;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_3008;
    }
    //fungsi mencetak SLL

```

```

public static void printList_3008(NodeSLL_2511533008 head_3008) {
    NodeSLL_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        System.out.print(curr_3008.data_3008 + "-->");
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008.next_3008 == null) {
        System.out.print(curr_3008.data_3008);
        System.out.println();
    }
}

public static void main(String[] args) {
    //buat SLL 1->2->3->4->5->6->null
    NodeSLL_2511533008 head_3008 = new NodeSLL_2511533008(1);
    head_3008.next_3008 = new NodeSLL_2511533008(2);
    head_3008.next_3008.next_3008 = new NodeSLL_2511533008(3);
    head_3008.next_3008.next_3008.next_3008 = new
NodeSLL_2511533008(4);
    head_3008.next_3008.next_3008.next_3008.next_3008 = new
NodeSLL_2511533008(5);
    head_3008.next_3008.next_3008.next_3008.next_3008.next_3008 =
new NodeSLL_2511533008(6);
    //cetak list awal
    System.out.println("List awal : ");
    printList_3008(head_3008);
    //hapus head
    head_3008 = deleteHead_3008(head_3008);
    System.out.println("List setelah head dihapus: ");
    printList_3008(head_3008);
    //hapus node terakhir
    head_3008 = removeLastNode_3008(head_3008);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList_3008(head_3008);
    //deleting node at position 2
    int position_3008 = 2;
    head_3008 = deleteNode_3008(head_3008, position_3008);
    //print list after deletion
    System.out.println("List setelah posisi 2 dihapus: ");
    printList_3008(head_3008);
}
}

```

Penjelasan:

1. Fungsi deleteHead_3008

```
//fungsi untuk menghapus head
public static NodeSLL_2511533008 deleteHead_3008(NodeSLL_2511533008 head_3008) {
    //jika SLL kosong
    if (head_3008 == null)
        return null;
    //pindahkan head ke node berikutnya
    head_3008 = head_3008.next_3008;
    //return head baru
    return head_3008;
}
```

Gambar 2.2.4.1 fungsi 1

Fungsi ini digunakan untuk menghapus node pertama (head).

2. Fungsi removeLastNode_3008

```
//fungsi menghapus node terakhir SLL
public static NodeSLL_2511533008 removeLastNode_3008(NodeSLL_2511533008 head_3008) {
    //jika list kosong, return null
    if (head_3008 == null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_3008.next_3008 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511533008 secondLast_3008 = head_3008;
    while (secondLast_3008.next_3008.next_3008 != null) {
        secondLast_3008 = secondLast_3008.next_3008;
    }
    //hapus node terakhir
    secondLast_3008.next_3008 = null;
    return head_3008;
}
```

Gambar 2.2.4.2 fungsi 2

Fungsi ini digunakan untuk menghapus node terakhir.

3. Fungsi deleteNode_3008

```
//fungsi menghapus node di posisi tertentu
public static NodeSLL_2511533008 deleteNode_3008(NodeSLL_2511533008 head_3008, int position_3008) {
    NodeSLL_2511533008 temp_3008 = head_3008;
    NodeSLL_2511533008 prev_3008 = null;
    //jika linked list null
    if (temp_3008 == null) |
        return head_3008;
    //kasus 1: head dihapus
    if (position_3008 == 1) {
        head_3008 = temp_3008.next_3008;
        return head_3008;
    }
    //kasus 2: menghapus node ditengah
    //telusuri ke node yang dihapus
    for (int i_3008 = 1; temp_3008 != null && i_3008 < position_3008; i_3008++) {
        prev_3008 = temp_3008;
        temp_3008 = temp_3008.next_3008;
    }
    //jika ditemukan, hapus node
    if (temp_3008 != null) {
        prev_3008.next_3008 = temp_3008.next_3008;
    } else {
        System.out.println("Data tidak ada");
    }
    return head_3008;
}
```

Gambar 2.2.4.3 fungsi 3

Fungsi ini digunakan untuk menghapus node pada posisi tertentu.

4. Fungsi printList_3008


```

//fungsi mencetak SLL
public static void printList_3008(NodeSLL_2511533008 head_3008) {
    NodeSLL_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        System.out.print(curr_3008.data_3008 + "-->");
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008.next_3008 == null) {
        System.out.print(curr_3008.data_3008);
        System.out.println();
    }
}

```

Gambar 2.2.4.4 fungsi 4

Fungsi ini digunakan untuk mencetak isi list.

5. Algoritma utama

```

public static void main(String[] args) {
    //buat SLL 1->2->3->4->5->6->null
    NodeSLL_2511533008 head_3008 = new NodeSLL_2511533008(1);
    head_3008.next_3008 = new NodeSLL_2511533008(2);
    head_3008.next_3008.next_3008 = new NodeSLL_2511533008(3);
    head_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(4);
    head_3008.next_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(5);
    head_3008.next_3008.next_3008.next_3008.next_3008.next_3008 = new NodeSLL_2511533008(6);
    //cetak list awal
    System.out.println("List awal : ");
    printList_3008(head_3008);
    //hapus head
    head_3008 = deleteHead_3008(head_3008);
    System.out.println("List setelah head dihapus: ");
    printList_3008(head_3008);
    //hapus node terakhir
    head_3008 = removeLastNode_3008(head_3008);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList_3008(head_3008);
    //deleting node at position 2
    int position_3008 = 2;
    head_3008 = deleteNode_3008(head_3008, position_3008);
    //print list after deletion
    System.out.println("List setelah posisi 2 dihapus: ");
    printList_3008(head_3008);
}
}

```

Gambar 2.2.4.5 algoritma utama

Algoritma utama ini pertama membuat list awal, lalu cetak list awal, hapus head, hapus node terakhir, dan hapus node pada posisi ke 2.

2.3 Output Program

2.3.1 Output TambahSLL_2511533008

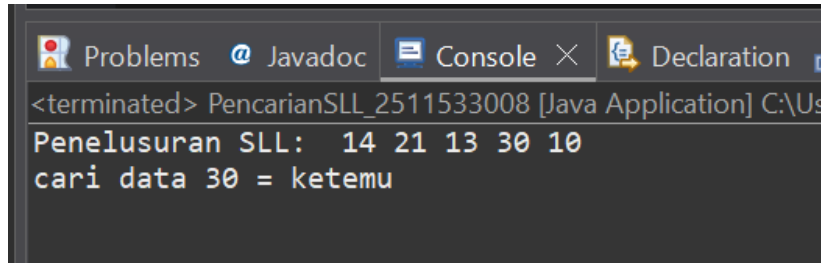
```

<terminated> TambahSLL_2511533008 [Java Application] C:\Users\asusvivobook
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

Gambar 2.3.1 output 1

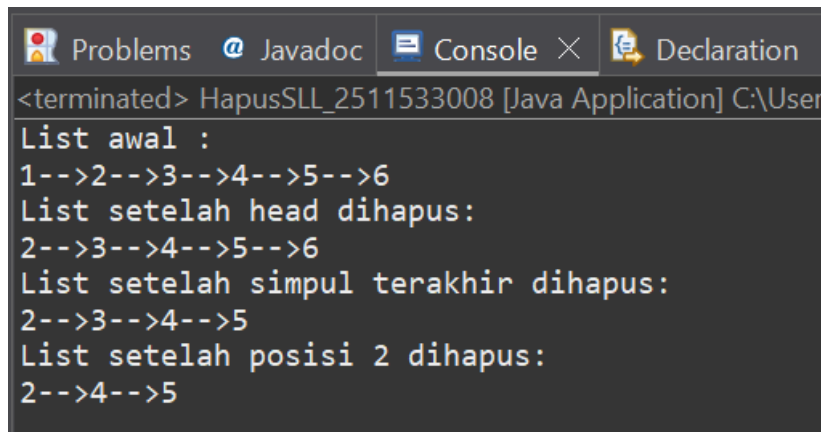
2.3.2 Output PencarianSLL_2511533008



```
<terminated> PencarianSLL_2511533008 [Java Application] C:\Us
Penelusuran SLL: 14 21 13 30 10
cari data 30 = ketemu
```

Gambar 2.3.2 output 2

2.3.3 Output HapusSLL_2511533008



```
<terminated> HapusSLL_2511533008 [Java Application] C:\User
List awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.3.3 output 3

BAB III

PENUTUP

3.1 Kesimpulan

Melalui praktikum struktur data Single Linked List, kita dapat memahami bahwa linked list merupakan salah satu struktur data dinamis yang memungkinkan pengelolaan data secara fleksibel melalui simpul (node) yang saling terhubung dengan pointer. Implementasi operasi dasar seperti traversal, insertion, deletion, dan searching memberikan pengalaman langsung dalam mengaplikasikan konsep teori ke dalam kode program, khususnya menggunakan bahasa Java. Praktikum ini juga menegaskan keunggulan Single Linked List dibandingkan array dalam hal efisiensi penambahan dan penghapusan data, meskipun tetap memiliki keterbatasan pada akses data yang harus dilakukan secara sekuensial. Dengan demikian, praktikum ini tidak hanya memperkuat pemahaman kita terhadap konsep dasar struktur data, tetapi juga membekali kita dengan keterampilan analitis dan teknis yang menjadi fondasi untuk mempelajari struktur data yang lebih kompleks serta mendukung pengembangan perangkat lunak di masa depan.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.
- [2] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.
- [3] Universitas Andalas, "Linked List," Fakultas Teknologi Informasi, 2026.

TUGAS

1. Class Pasien_2511533008

Kode Program:

```
package pekan5_2511533008;

public class Pasien_2511533008 {
    private String namaPasien_3008;
    private String penyakit_3008;
    private int nomorAntrian_3008;
    private Pasien_2511533008 next_3008;

    public Pasien_2511533008(String nama_3008, String penyakit_3008, int
nomor_3008) {
        this.namaPasien_3008 = nama_3008;
        this.penakit_3008 = penyakit_3008;
        this.nomorAntrian_3008 = nomor_3008;
        this.next_3008 = null;
    }

    public String getNamaPasien_3008() { return namaPasien_3008; }
    public String getPenyakit_3008() { return penyakit_3008; }
    public int getNomorAntrian_3008() { return nomorAntrian_3008; }
    public Pasien_2511533008 getNext_3008() { return next_3008; }

    public void setNext_3008(Pasien_2511533008 next) { this.next_3008 =
next; }
}
```

2. Class RumahSakit_2511533008

Kode Program:

```
package pekan5_2511533008;
import java.util.Scanner;

public class RumahSakit_2511533008 {
    private Pasien_2511533008 head_3008;
    private int counter_3008;

    public RumahSakit_2511533008() {
        head_3008 = null;
        counter_3008 = 0;
    }

    // Daftarkan Pasien (Insert at Tail)
    public void daftarkanPasien_3008(String nama_3008, String
penyakit_3008) {
```

```

        counter_3008++;
        Pasien_2511533008 baru_3008 = new Pasien_2511533008(nama_3008,
penyakit_3008, counter_3008);
        if (head_3008 == null) {
            head_3008 = baru_3008;
        } else {
            Pasien_2511533008 temp_3008 = head_3008;
            while (temp_3008.getNext_3008() != null) {
                temp_3008 = temp_3008.getNext_3008();
            }
            temp_3008.setNext_3008(baru_3008);
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
" + counter_3008);
    }

    // Panggil Pasien (Delete Head)
    public void panggilPasien_3008() {
        if (head_3008 == null) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Memanggil pasien: " +
head_3008.getNamaPasien_3008() +
                " | Keluhan: " +
head_3008.getPenyakit_3008());
            head_3008 = head_3008.getNext_3008();
        }
    }

    // Tampilkan Antrian (Display)
    public void tampilkanAntrian_3008() {
        if (head_3008 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        Pasien_2511533008 temp_3008 = head_3008;
        while (temp_3008 != null) {
            System.out.println("No: " + temp_3008.getNomorAntrian_3008()
+
                " | Nama: " +
temp_3008.getNamaPasien_3008() +
                " | Keluhan: " +
temp_3008.getPenyakit_3008());
            temp_3008 = temp_3008.getNext_3008();
        }
    }

    // Cari Pasien (Search by name, case-insensitive)
    public void cariPasien_3008(String nama_3008) {
        Pasien_2511533008 temp_3008 = head_3008;
        boolean found_3008 = false;
        while (temp_3008 != null) {

```

```

        if
(temp_3008.getNamaPasien_3008().equalsIgnoreCase(nama_3008)) {
            System.out.println("Pasien ditemukan: " +
temp_3008.getNamaPasien_3008() +
                " | Keluhan: " +
temp_3008.getPenyakit_3008() +
                " | No Antrian: " +
temp_3008.getNomorAntrian_3008());
            found_3008 = true;
            break;
        }
        temp_3008 = temp_3008.getNext_3008();
    }
    if (!found_3008) {
        System.out.println("Pasien dengan nama " + nama_3008 + "
tidak ditemukan.");
    }
}

// Cek Status Antrian
public void cekStatusAntrian_3008() {
    if (head_3008 == null) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Total pasien: " + counter_3008);
        System.out.println("Pasien terdepan: " +
head_3008.getNamaPasien_3008());
    }
}

public static void main(String[] args) {
    RumahSakit_2511533008 rs = new RumahSakit_2511533008();
    Scanner sc_3008 = new Scanner(System.in);
    int pilihan_3008;

    do {
        System.out.println("\n=== MENU ANTRIAN RUMAH SAKIT ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("0. Keluar");
        System.out.print("Pilihan: ");
        pilihan_3008 = sc_3008.nextInt();
        sc_3008.nextLine(); // buang newline

        switch (pilihan_3008) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_3008 = sc_3008.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String penyakit_3008 = sc_3008.nextLine();
                rs.daftarkanPasien_3008(nama_3008, penyakit_3008);

```

```

        break;
    case 2:
        rs.panggilPasien_3008();
        break;
    case 3:
        rs.tampilkanAntrian_3008();
        break;
    case 4:
        System.out.print("Masukkan Nama Pasien yang dicari:
");
        String cari_3008 = sc_3008.nextLine();
        rs.cariPasien_3008(cari_3008);
        break;
    case 5:
        rs.cekStatusAntrian_3008();
        break;
    case 0:
        System.out.println("Program selesai. Terima
kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
} while (pilihan_3008 != 0);

sc_3008.close();
}
}

```


3. Output Tugas

```
RumahSakit_2511533008 [Java Application] C:\Users\asusvivobook\p2\pool\plugins
=== MENU ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
0. Keluar
Pilihan: 1
Masukkan Nama Pasien: Yudhistira Patibrata
Masukkan Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== MENU ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
0. Keluar
Pilihan: 1
Masukkan Nama Pasien: Arvasetya
Masukkan Keluhan: Batuk,Pilek
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== MENU ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
0. Keluar
Pilihan: 3
No: 1 | Nama: Yudhistira Patibrata | Keluhan: Demam
No: 2 | Nama: Arvasetya | Keluhan: Batuk,Pilek

=== MENU ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
0. Keluar
Pilihan: 5
Total pasien: 2
Pasien terdepan: Yudhistira Patibrata

=== MENU ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
0. Keluar
Pilihan: 2
Memanggil pasien: Yudhistira Patibrata | Keluhan: Demam
```

Gambar output tugas